v0

Requirements Document — Blog Simple Backend

Project: Blog Simple

Technologies: FastAPI, SQLAlchemy, PostgreSQL

Author: Sahira Tejada

Version: 1.0

Date: 2025-10-07

---

1. Purpose

The purpose of this document is to describe the functional and non-functional requirements of the Blog Simple backend system. This API will enable users to create and interact with blog posts, comments, likes, categories, and user relationships.

2. Scope

The system will provide the backend functionality for a blogging platform that supports:

- User registration, authentication, and profile management.
- Post creation, editing, listing, and deletion.
- Commenting and nested replies.
- Following/unfollowing users.
- Liking posts.
- Searching and filtering posts by category.

The frontend, notifications, and analytics are **out of scope** for this phase.

3. Definitions

Term	Definition
API	Application Programming Interface for communication between frontend and backend.
JWT	JSON Web Token, used for authentication.
ORM	Object Relational Mapper (SQLAlchemy).
CRUD	Create, Read, Update, Delete operations.
ERD	Entity Relationship Diagram.

4. Functional Requirements

4.1 User Management

Description: Allow users to register, authenticate, and manage their profiles.

- FR-1: The system must allow user registration with unique email and username.
- FR-2: Passwords must be securely hashed using bcrypt or argon2.
- FR-3: The system must provide JWT authentication for login.
- FR-4: Users can view and update their profiles.

- FR-5: Users can follow or unfollow other users.

4.2 Post Management

Description: Manage creation and publication of blog posts.

- FR-6: Authenticated users can create, edit, and delete their own posts.
- FR-7: Posts contain title, content, author, categories, timestamps.
- FR-8: Posts can be published or saved as draft (optional future extension).
- FR-9: Users can like or unlike posts.
- FR-10: Posts can be searched by title, content, or filtered by category.

4.3 Comment Management

Description: Allow users to comment and reply to posts.

- FR-11: Users can comment on posts.
- FR-12: Users can reply to existing comments (nested comments up to 2 levels).
- FR-13: Comments display author, timestamp, and content.
- FR-14: Only comment author or admin can delete a comment.

4.4 Categories

Description: Categorize posts for filtering.

- FR-15: Admins can create, edit, or delete categories.
- FR-16: Each post can have multiple categories.
- FR-17: Users can filter posts by category slug.

4.5 Likes and Follows

Description: Support simple engagement features.

- FR-18: Users can like/unlike posts (toggle behavior).
- FR-19: Users can follow/unfollow other users.
- FR-20: Display counts for likes and followers.

4.6 Role-base

Description: Define which roles can perform each action in the system to ensure security and proper functionality.

- **Guests:** Can view public posts, comments, user profiles, and search/filter posts.
- **Users:** Can perform all actions available to Guests plus create, edit, delete their own posts and comments, like/unlike posts, follow/unfollow other users.
- **Admins:** Full access to manage posts, comments, categories, users; can delete any post or comment.

Role-based access matrix:

Functionality	Guest	User	Admin
View posts and comments	✅	✅	✅
Create posts	❌	✅	✅
Edit/delete own posts	❌	✅	✅
Delete posts by others	❌	❌	✅
Comment on posts	❌	✅	✅
Reply to comments	❌	✅	✅
Delete comments by others	❌	❌	✅
Create/edit/delete categories	❌	❌	✅
Like/unlike posts	❌	✅	✅
Follow/unfollow users	❌	✅	✅
View user profiles, followers/following	✅	✅	✅

5. Non-Functional Requirements

ID	Requirement	Description
NFR-1	Performance	API should respond in <300ms for common queries.
NFR-2	Security	Use HTTPS, hashed passwords, JWT, input validation.
NFR-3	Scalability	The system must support at least 20 users and 100 posts.

ID	Requirement	Description
NFR-4	Availability	99% uptime target under normal load.
NFR-5	Maintainability	Code must follow clean architecture with clear separation of layers.
NFR-6	Testability	At least 80% unit test coverage using pytest.
NFR-7	Portability	The system must run on Dockerized containers.

6. User Roles and Permissions

Role	Permissions
Guest	Can view public posts and comments.
User	Can create/edit/delete own posts and comments, like posts, follow users.
Admin	Full access to manage posts, comments, categories, and users.

7. System Interfaces

- **Database:** PostgreSQL for relational storage.
- **API:** FastAPI for RESTful services.
- **Authentication:** OAuth2 password grant with JWT tokens.
- **External services (optional):** Redis for caching, Celery for background tasks.

8. Data Requirements

- Each entity must have UUID as primary key.
- Timestamps for creation and modification.
- Foreign key constraints enforced for relationships.
- Use Alembic migrations to manage schema versions.

9. Error Handling & Validation

- Invalid inputs return `422 Unprocessable Entity`.
- Unauthorized access returns `401 Unauthorized`.

- Forbidden actions return `403 Forbidden`.
- Missing entities return `404 Not Found`.
- Internal errors return `500 Internal Server Error`.

---

10. Future Enhancements

- Role-based access control.
- Real-time notifications (WebSocket).
- Post versioning and drafts.
- User activity feed.
- Search engine integration (ElasticSearch).

---

11. Acceptance Criteria

- All endpoints return correct HTTP status codes.
- All CRUD operations function with valid authentication.
- The system passes functional and integration tests.
- Documentation (OpenAPI schema) auto-generated from FastAPI.

---

12. References

- FastAPI official docs: https://fastapi.tiangolo.com
- SQLAlchemy ORM docs: https://docs.sqlalchemy.org
- PostgreSQL docs: https://www.postgresql.org/docs
- OWASP API Security Top 10